

How the machine actually works

Every phase of this pipeline is started by one thing: **a label appearing on an issue or a pull request**. There is no scheduler driving the flow, no queue, no orchestrator process. The state lives in GitHub's labels, and the transitions are shell scripts.

This page is the mechanical explanation — what fires what, and which technique defends against which failure. Everything here was extracted from the repository itself, not from memory: 9 workflows, 10 jobs, 74 distinct guards.

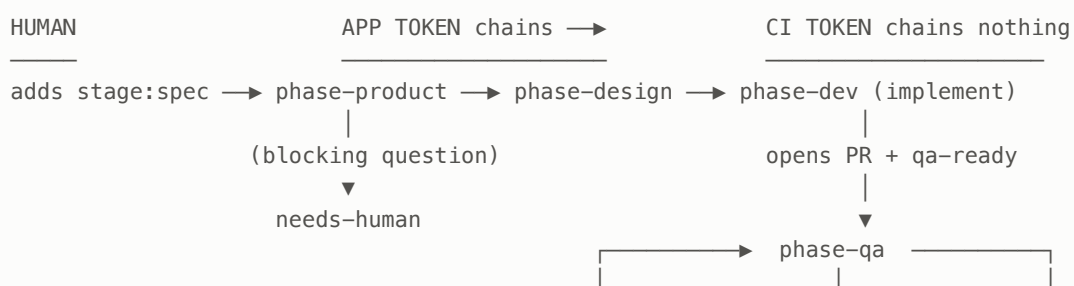
The one rule that makes it safe

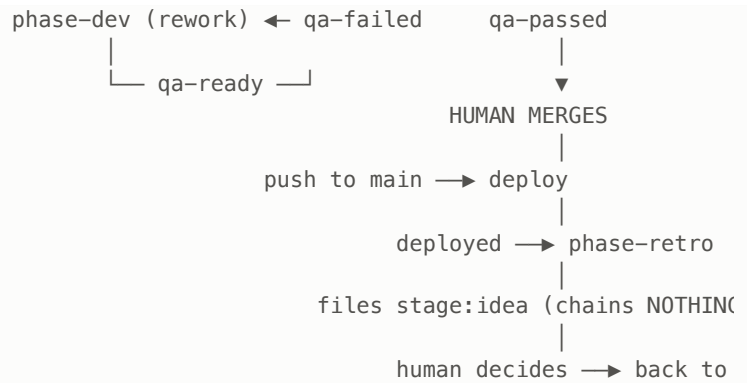
Label events written by the GitHub App chain to the next phase. Label events written by the default CI token do not.

GitHub deliberately suppresses workflow events caused by `GITHUB_TOKEN`, to stop CI from triggering itself forever. This pipeline turns that suppression into a design tool: anything that *should* advance the state machine is written with the App token; anything that must *not* — commit statuses, cost comments, verdict comments, retro proposals — is written with the ordinary CI token. The asymmetry is the engine.

The cost of that power is that App-token events get **no** recursion suppression at all, so loop safety is hand-built: every phase is name-guarded to a single label, every transition removes the inbound label in the same step that adds the outbound one, a timeline counter aborts a runaway ticket, and a lint proves the label graph is acyclic apart from one sanctioned cycle.

Triggers — what fires what





ci ← pull_request (any) · push to main
 nightly-fuzz ← schedule 03:17

- required status
- never gates a PR

JOB	FIRES ON	ON SUCCESS IT WRITES	TOKEN
product spec + draft ACs	issues.labeled == stage:spec	→ stage:design , or parks at needs-human if the spec carries a <i>blocking</i> open question	App
design final AC-1..n	issues.labeled == stage:design	→ stage:dev	App
implement branch, code, tests, PR	issues.labeled == stage:dev	opens a PR with Fixes #N , labels it qa-ready , moves the issue → stage:qa	App
qa tier-1 suites + agent	pull_request.labeled == qa-ready	a per-SHA qa/agent-verdict status, then → qa-passed / qa-failed / needs-human	App
rework fix the findings	pull_request.labeled == qa-failed	pushes commits, → qa-ready (max 3 cycles)	App
ci lint, tests, typecheck	pull_request (any) push to main	a required status; blocks the merge button	CI
deploy gateway + close-out	push to main	issue → stage:done + deployed	App
retro advisory analyst	issues.labeled == deployed	a retro comment + ≤2 stage:idea proposals — deliberately on the CI token, so they chain nothing	CI
nightly-fuzz unbounded property fuzzing	schedule 03:17	files a bug on failure; never gates a PR	CI

Two moments are human by construction: applying the entry label (which starts spend) and pressing merge (which ships code). Everything between them runs unattended.

Techniques, by what they defend against

Making a model's output safe to act on

AGAINST: PARSING FREE TEXT FOR CONTROL FLOW

- **Schema-validated structured output.** Every phase returns JSON validated against a file in `ci/claude/schemas/`. No structured output $\Rightarrow$ the phase exits non-zero and the ticket parks. Free text is never parsed for a decision.
- **The model never chooses a transition.** Agents produce *content*; deterministic YAML and shell decide what happens next. This is the design thesis the whole experiment tests.
- **Evidence over assertion.** A QA `fail` without reproduction steps is downgraded to `blocked` automatically — an unevidenced failure parks for a human instead of triggering rework.
- **Substitution must be declared.** If QA verifies an acceptance criterion with an easier scenario, it must say so and name the tier-1 test covering the literal case.

Making the gate honest

AGAINST: A GREEN LIGHT THAT HASN'T EARNED IT

- **Two tiers, one deterministic decision.** Tier 1 is ordinary test suites; tier 2 is the agent. The gate rule is in YAML: tier-1 green *and* agent verdict `pass` $\Rightarrow$ pass. **Any tier-1 failure outranks the agent's opinion** — that has fired for real, on a run where the agent argued the failure was out of scope.
- **Statuses are per-SHA.** Push after QA and the new commit has no verdict, so the merge re-blocks by construction. New code must earn a new verdict.
- **Branch protection requires both `ci` and `qa/agent-verdict`.** The merge button is a human gate over machine-verified state.
- **Determinism where it's claimed.** The contract tier no longer generates random inputs at gate time — it replays a committed corpus, and a schema-digest drift check fails loudly. Unbounded fuzzing moved to a nightly job that gates nothing.

Keeping agents inside their lane

AGAINST: AN AGENT EDITING THE RULES THAT GOVERN IT

- **Three layers, deliberately redundant** — see below. The pipeline's own charters, scripts and workflows are off-limits to every agent.
- **Credential-less checkouts.** Agent steps hold no token at all; only the orchestrator's own push steps authenticate.
- **No `workflows: write`.** The App physically cannot alter workflow files — that boundary is enforced by GitHub, not by us.

- **Invariant tests over the profiles themselves.** A test fails the build if any phase re-introduces an arg-scoped shell allowlist, if an analysis phase gains a shell, or if a build phase loses one.

Not paying twice for the same fact

AGAINST: BURNING BUDGET REDISCOVERING WHAT WE ALREADY KNOW

- **Merge-ref freshness assert.** GitHub computes a PR's merge ref lazily and a label event doesn't refresh it, so QA could test a stale base. The job now compares the merge commit's parent to the live `main` tip and refuses to spend if they differ.
- **Repeat-failure guard.** Signs each run with the tested *tree*, the suite outcomes and the exact failing tests. An identical failure on an identical tree that was already triaged as environmental skips the agent and escalates to a human. It can only ever move the gate toward a human — never toward a pass.
- **Capacity precheck.** Counts collected e2e tests before the stack boots and compares demand against the gateway's request budget, so suite growth can't masquerade as product failures.

Bounding cost and runaway

AGAINST: AN AGENT LOOPING, OR A TICKET LOOPING

- **Per-phase turn and dollar caps.** Analysis phases get 30 turns / \$3; build phases 120 turns and \$8–12. The QA cap is now a repo variable, so raising it for an unusually large ticket is an operational decision rather than a code change.
- **Loop guards on the timeline.** A counter over label events aborts past a ceiling — 20 for a ticket overall, and just 3 for the QA↔rework cycle, which is deliberately tight.
- **Concurrency groups per ticket** and job timeouts, so two runs of the same phase can't race.
- **Triage rule, written down:** on a turn-cap overrun, read the permission denials *first*. Repeated denials mean a stuck agent — fix the sandbox. Zero denials with steady progress means the work outgrew the cap. Never raise the ceiling on a stuck agent.

Letting the system improve itself, safely

AGAINST: A SELF-MODIFYING LOOP WITH NO HUMAN IN IT

- **The retro phase is advisory.** It reads a shipped ticket's whole trajectory and drafts the lesson plus ≤ 2 proposals. It blocks nothing, transitions nothing, and a failure posts a comment instead of parking the ticket.
- **The loop cannot close itself.** Proposals are filed on the CI token as `stage:idea`, which triggers no phase. They enter the pipeline only when a human applies the entry label.
- **It dedupes against the open backlog** — an earlier version filed the same idea three times because it had no memory across runs.

- **Cost telemetry everywhere.** Every phase posts turns and dollars onto the ticket, and a metrics script computes lead time, rework, parks and spend straight from GitHub events — no surveys anywhere in the measurement path.

The privileged-path boundary, in three layers

Worth showing separately, because the ordering is the lesson: the first two layers are advisory and the third is the one that actually holds. An agent once edited the QA agent's own charter by walking straight through the first two.

LAYER 1 ·
PROMPT

The skill says don't.

Both dev skills name `.github/`, `ci/claude/` and `scripts/` as off-limits and tell the agent to escalate such findings instead. Advisory. One skill lacked the sentence, and its agent edited the charter.

LAYER 2 ·
PERMISSION

Deny rules block the edit tools.

Repo-relative `Edit()` and `Write()` denials on those paths. Advisory in practice: the patterns were once written with a leading `//`, which means filesystem-root, so every rule silently matched nothing.

LAYER 3 ·
DETERMINISTIC

A diff check before every push.

If any new agent commit touches those paths — or the working tree under them is dirty, since a child process can write without committing — the job fails and the ticket parks. This is the one that holds. It cannot be talked out of it.

Every guard has an incident behind it

Almost nothing here was designed up front. The pipeline ran, something went wrong, and the fix became a mechanism. This table is the scar tissue — and it's the most transferable part of the project, because these failures are not specific to this app.

WHAT HAPPENED

A QA agent burned 100 turns fighting its own sandbox — arg-scoped shell rules auto-denied every compound command it wrote

WHAT NOW EXISTS BECAUSE OF IT

Broad shell + a deny-list for build phases, and the **denials-first triage rule**

The same defect was later found still live in a sibling phase, costing a full run	Invariant tests over every phase's tool profile
A rework agent edited the QA agent's own charter, because the deny rules were silently dead	The three-layer boundary and its deterministic third layer
QA spent a full run testing a merge ref computed against stale <code>main</code>	The freshness assert — which later prevented an identical waste, at zero cost
Two consecutive QA runs paid full price to reach the same conclusion about the same environmental flake	The repeat-failure guard
The e2e suite outgrew the gateway's fixed request budget; 429s looked like product bugs	The capacity precheck , anchored on the measured breaking point
A green contract run passed on byte-identical code that had failed minutes earlier	Corpus replay — determinism by construction rather than by configuration
Dev-phase cost never appeared on any ticket; the retro flagged the symptom three times without finding the cause	The missing write permission , and telemetry that warns instead of failing silently
A human answer to a spec's open question raced the design phase that was already running	Blocking questions park the ticket instead of advancing
Four separate retro runs filed the same proposal	Retro dedupes against the open backlog, capped at two

If you're joining this

Three things explain most of the design:

Prose is not a boundary. Every rule that lives only in an agent's instructions will eventually be walked through — not maliciously, just by an agent doing its job under different wording. If a rule matters, it needs a deterministic step that fails the build.

Verify against the environment, not the description. Guards written by reasoning about behaviour have been wrong roughly as often as they've been right; guards verified by running them against real artifacts have held. Two separate guards in this repo were themselves broken on arrival and only caught by executing them.

Cheap to reverse is the useful question. Not "is this important" but "what does it cost to undo." That distinction decides which questions an agent may settle alone and which must stop and wait for a person — and it's the one the pipeline is still learning to encode.

Extracted from the repository at github.com/0legKonyk/agentik-development — 9 workflows, 10 jobs, 74 guards. The binding specification is [docs/sdlc.md](#) ; the incident record is [docs/learning-log.md](#) .